scope_enumerate.js — guide

Revision: 1 **Last modified:** 2026-09-26T13:34:00Z

Source: `constitution/scripts/codegraph/scope_enumerate.js` (254 lines). Tests: no test file of its own; it is exercised through `codegraph_scope_guard.py` by `constitution/scripts/codegraph/tests/run_scope_tests.sh` (cases T01-T17, mutants M0-M15; the scan-mode probe is case T17). Citations below are `scope_enumerate.js:<line>` unless another file is named.

Overview

A read-only bridge into an installed CodeGraph runner's OWN file-discovery code (header, lines 3-10). It loads the runner's `project-config.js`, `extraction/index.js` and `extraction/grammars.js` from a `lib/dist` directory (lines 87-98) and answers three questions with one JSON object on stdout:

Mode	Question it answers	Source
`enumerate`	Which files would the runner's `scanDirectory` index for this root?	lines 132-165
`contract`	Does the runner still behave the way the scope tooling assumes (config file name, exclude handling, built-in skips, `.gitignore` negation)?	lines 170-212
`explain`	Which tracked source files were dropped by the runner's built-in skips (as opposed to an explicit `exclude`)?	lines 215-249

It exists so the scope guard measures the engine's real behaviour instead of a re-implementation of it (lines 6-8). Its only caller in the tree is `codegraph_scope_guard.py` (`ENUM_JS`, `node_bridge`, `codegraph_scope_guard.py:61, 98-108`); it is not meant to be run by hand except to debug that guard.

Prerequisites

- `node` >= 18 and `git` (header line 26; `git` is called through `execFileSync`, line 107-109).
- A runner `lib/dist` tree containing `project-config.js`, `extraction/index.js` and `extraction/grammars.js` (lines 90-92). Any missing file is a "runner not loadable" refusal (exit 3, line 94).
- Optional: the `ignore` npm package resolvable from the dist tree; used by `explain` (line 229, hard failure if absent) and looked up (but not used) by `loadRunner` (line 96, failure tolerated).

Usage

```
node scope_enumerate.js <mode> --dist <runner lib/dist> [--root <dir>] [opts]
```

The first argument is the mode; every following argument must be a `--key value` PAIR (lines 41-50). A key without a value, or a non-`--` token where a key is expected, is refused with exit 3 (`bad argument near <key>`, line 46). `--dist` is required for every mode (line 53).

Option	Modes	Meaning	Source
`--dist DIR`	all	Runner `lib/dist`. Never modified	lines 53-54
`--root DIR`	enumerate, explain	Project root. Required by both	lines 133, 216
`--out FILE`	enumerate, explain	Where the file list / dropped-groups table is written. Required	lines 133, 216
`--config-filename NAME`	all	Runner's project-config file name. Default `codegraph.json`	line 55
`--overlay-config FILE`	enumerate, explain	Answer reads of `<root>/<config-filename>` from FILE instead of disk	lines 63, 134, 217
`--overlay-gitignore FILE`	enumerate, explain	Answer reads of `<root>/.gitignore` from FILE instead of disk	lines 64, 134, 217
`--roots-file FILE`	enumerate	JSON array of project-relative directories; the answer then carries `root_census`	lines 160-163
`--work DIR`	contract	Scratch directory for the throwaway probe repository. Required	line 171
`--enumerated FILE`	explain	A previously written enumeration (`--out` of `enumerate`). Required	line 216

Overlays work by replacing `fs.readFileSync`, `fs.statSync` and `fs.existsSync` on the shared `fs` module BEFORE the runner is required, so reads of exactly the two overlaid absolute paths are answered from the overlay files and every other read goes to disk (lines 57-85). Nothing is written to the project.

Examples

All examples were run against a throwaway git repository in a scratch directory (one tracked `src/a.c`, one tracked `build/b.c`) and the installed runner dist printed by `codegraph_scope_guard.py --resolve-dist` (`.../codegraph-linux-x64/lib/dist`, runner version 1.6.0). The real repository and its `.codegraph` were never touched.

#	Command	Status
1	`node scope_enumerate.js` (no arguments)	VERIFIED (ran; saw `{"ok":false,"error":"--dist <runner lib/d`
2	`node scope_enumerate.js bogus --dist <dist>`	VERIFIED (ran; saw `{"ok":false,"error":"unknown mode bogus"}`
3	`node scope_enumerate.js enumerate --dist / nonexistent --root <p> --out <f>`	VERIFIED (ran; saw `{"ok":false,"error":"runner not loadable`
4	`node scope_enumerate.js enumerate --dist <dist> --root` (key without value)	VERIFIED (ran; saw `bad argument near --root`, exit 3)
5	`node scope_enumerate.js enumerate --dist <dist> --root <p> --out <f>`	VERIFIED (ran; saw `{"ok":true,"mode":"enumer-ate","count":1,"ms":21,"scan_mode":"git","runner_version":"1 []}`, exit 0; `<f>` held exactly `src/a.c`; `build/b.c` was dropped b
6	Same plus `--roots-file` containing `["src","nothere"]`	VERIFIED (ran; saw `root_census` `{"src":{"checked_out":false,` directory is not its own git repository, so it is "not checked out" in this se
7	`node scope_enumerate.js contract --dist <dist> --work <scratch>`	VERIFIED (ran; saw `"ok":true` with six checks C0-C5 all `ok:true`,

#	Command	Status
8	`node scope_enumerate.js explain --dist <dist> --root <p> --enumerated <f> --out <t>`	VERIFIED (ran; saw `{"ok":true,"mode":"explain","tracked_sour` `fault:build/`)
9	`enumerate` with `--overlay-config` naming a file `{"ex-clude":["/src/"]}`	VERIFIED (ran; saw `count":0` and `overlay` listing `<root>/codegr` no `codegraph.json` was written to the project)
10	`contract` mode with a broken runner (exit 2)	NOT RUN (no drifted runner available; the exit path is `process.exit(`
11	`enumerate` against this repository's real root	NOT RUN (the brief forbids pointing anything at the live 580K-file tree)

Outputs

One JSON object on stdout, always the last line (the guard reads the last non-empty stdout line, `codegraph_scope_guard.py:103`), plus any file named by `--out` .

- Refusals: `{"ok":false,"error":"<message>"}` (`die` , lines 36-39).
- `enumerate` (lines 155-164): `ok` , `mode` , `count` , `ms` , `scan_mode` , `runner_version` , `runner_config_filename` , `overlay` (list of overlaid absolute paths) and, with `--roots-file` , `root_census` = `{dir: {checked_out, tracked_source}}` . `--out` receives the sorted file list, one path per line, plus a trailing newline unless the list is empty (lines 153-154).
- `contract` (lines 208-209): `ok` , `mode` , `runner_version` , `runner_config_filename` , `checks` = list of `{id, ok, detail}` .

- `explain` (line 248): `ok`, `mode`, `tracked_source`, `dropped`, `groups`; `--out` receives one row per group, `<count><TAB><key>`, sorted by count descending then key (lines 246-247). Keys are `default:<dir>/` for a directory matched by the runner's default ignore list, otherwise `non-default:<first two path segments>/` (lines 238-244).

Exit codes

Code	Meaning	Source
0	Success (`enumerate`, `explain`, and `contract` when every check passed)	lines 164, 211, 248
2	`contract` only: at least one check failed (runner drift)	line 211
3	Usage error, unknown mode, missing required option, or runner not loadable	lines 36-39, 46, 53, 94, 133, 171, 216, 254

An uncaught exception (for example `git` failing inside `explain`, or an unreadable `--roots-file`) is not turned into a `die`: Node prints its own stack trace to stderr and exits 1. UNCONFIRMED: not run; read from the code, which has no surrounding `try` for those calls.

Edge cases

- **Scan mode is observed, not assumed** (lines 136-149). For the duration of `scanDirectory` the script wraps `child_process.execFileSync`. If the runner's `git ls-files -z -s --recurse-submodules` (or `rev-parse --show-toplevel`) throws, the runner silently falls back to a filesystem walk; the wrapper records `scan_mode` as `walk_fallback:<code>` (or `walk_fallback:not_a_git_repo`) and re-throws so the runner behaves as it would unwrapped. Otherwise `scan_mode` is `git`. The wrapper is removed straight after the scan (line 152), but not if the scan itself throws.

- **`contract` recreates its probe repository** at `<work>/cg_contract_repo` every time (`rmSync` then `mkdirSync` , lines 173-175) and removes it at the end (line 210). It runs C0 (config filename equals `--config-filename`), C1 (the probe repository is visible at all: the control needle), C2 (`build/` is skipped by default), C3 (a config `exclude` is honoured and an unknown `_generated` key is tolerated), C4 (a root `.gitignore` negation `!/build/` re-includes it), C5 (an `exclude` beats the negation) (lines 188-206).
- **Census counts a root only if it is its own git repository.** `checked_out` is true only when `git rev-parse --show-toplevel` run inside the directory resolves to the directory itself (lines 121-124); a missing directory or an ordinary sub-directory of the parent repo is `checked_out:false, tracked_source:0` (example 6). `tracked_source` counts that repository's own tracked files (`git ls-files` , without `--recurse-submodules`) that the runner treats as source (lines 125-126).
- **`explain` does not count explicit excludes as drops** (line 235) and needs the `ignore` package (line 229).
- **Overlay is exact-path.** Only `<root>/<config-filename>` and `<root>/.gitignore` are answered from overlay files (lines 63-64, 67); a nested `.gitignore` is read from disk.

Internal behaviour

1. Parse `mode` and `--key value` pairs; require `--dist` (lines 41-55).
2. `enumerate` / `explain` : install the overlay (only if an overlay option was given), then load the runner (lines 134-135, 217-218); `contract` loads the runner without an overlay (line 172).
3. Do the mode's work (`scanDirectory` ; probe repository; or diff of `git ls-files --recurse-submodules` against the enumeration) and print one JSON object.

Related

File	Relation
code-graph_scope_guard	The only caller; judges the JSON this script returns
scope_render	Renders the `codegraph.json` / `.gitignore` block that the overlay options let a dry run apply
owed_report	Reports whether this tool has tests and a guide

Last verified

2026-09-26 (source lines read on that date; the examples above were run that day on a scratch fixture). Findings: none. The in-source header block (lines 2-29) matches the code; it does not state the `walk_fallback` values of `scan_mode`, which the code emits (line 138-146) and this guide documents.